



PRÉ-RENTRÉE 2026-2027

Guide du formateur

Stage Première NSI - maîtrise de Python

5 séances de 2 heures - 10 heures

Python 3 - théorie, pratique, tests et projet

Document enseignant confidentiel

1. Principes de conduite

1.1 Posture

- ne jamais supposer que la réponse à un QCM prouve une maîtrise pratique ;
- faire tracer avant d'exécuter ;
- demander « quelle valeur possède chaque variable à cet instant ? » ;
- faire écrire des tests avant de multiplier les exercices ;
- distinguer erreur de syntaxe, erreur d'exécution et erreur logique ;
- ne pas corriger immédiatement une certitude erronée : provoquer d'abord un contre-exemple ;
- valoriser un code simple, lisible et testé plutôt qu'un code court mais opaque.

1.2 Organisation matérielle

- un ordinateur par élève ;
- Python 3 installé, idéalement via Thonny ou un environnement équivalent ;
- aucun paquet externe requis ;
- dossier 06_CODE copié localement ;
- mode hors ligne possible ;
- vidéoprojecteur pour les revues de code ;
- feuilles de trace et cartes d'aide imprimées.

1.3 Rôles en binôme

Pour les activités communes, alterner toutes les 12 à 15 minutes :

- **pilote** : écrit le code ;
- **navigateur** : lit la spécification, anticipe le résultat, contrôle les tests ;
- puis inversion des rôles.

2. Diagnostic de départ

Ahmad BELDI

Le bilan montre une programmation encore fragile : 13 questions traitées sur 18, programmation à 22,2 %, avec des erreurs sur l'affectation, l'incrémentation, les négations, `range`, l'indexation, `len`, les paramètres et `None`. La priorité n'est pas la vitesse mais la construction d'un modèle mental correct.

Ahmed BENHADJ SALEM

Le bilan de Première montre quatre domaines solides mais une programmation à 55,6 %. Les erreurs portent sur la négation, le choix `for/while`, `len` et `None`. Un bilan de niveau Terminale met aussi en évidence une erreur d'accumulateur. L'enjeu est la fiabilité sous contrainte, pas la découverte théorique.

3. Routine de séance

1. rituel sans ordinateur ;
2. prédiction du comportement d'un code ;
3. exécution et comparaison ;
4. apport théorique court ;
5. programmation guidée ;
6. parcours différencié ;
7. tests et revue ;
8. exit ticket et mise à jour du tableau de bord.

4. Barème de compétences

Niveau	Description
0	non situé
1	ne sait pas commencer ou applique une conception erronée
2	réussit avec aide ou sur un cas familier
3	réussit seul, explique et teste

Niveau	Description
4	adapte à un problème nouveau et aide à valider une solution

5. Déroulé des séances

Séance 1

- **objectif commun** : comprendre l'affectation et écrire un programme conditionnel ;
- **Ahmad** : trace guidée, une variable par ligne, code à trous ;
- **Ahmed** : conditions composées, négations et cas limites ;
- **preuve attendue** : deux tables de trace exactes et un programme avec trois branches testé sur les bornes.

Séance 2

- **objectif commun** : maîtriser les boucles et les schémas compteur/accumulateur ;
- **Ahmad** : `range`, indices, compteur, somme ;
- **Ahmed** : `while`, terminaison, accumulateur, tests sur liste vide ;
- **preuve attendue** : programme calculant somme, moyenne, maximum et nombre de valeurs répondant à un critère.

Séance 3

- **objectif commun** : écrire des fonctions propres et testées ;
- **Ahmad** : paramètre, argument, `return`, `None` ;
- **Ahmed** : préconditions, postconditions, assertions et couverture de tests ;
- **preuve attendue** : module de quatre fonctions et au moins huit assertions.

Séance 4

- **objectif commun** : manipuler listes, dictionnaires et table CSV ;
- **Ahmad** : indexation, `len`, `append`, parcours ;
- **Ahmed** : aliasing, compréhensions, dictionnaires et validation des lignes ;
- **preuve attendue** : import CSV, filtre, tri et calcul statistique.

Séance 5

- **objectif commun** : résoudre un problème complet ;
- **Ahmad** : squelette guidé ;
- **Ahmed** : cahier des charges seul, tests supplémentaires et comparaison d'algorithmes ;
- **preuve attendue** : programme exécutable, documenté, testé et expliqué oralement.

6. Questions de relance

- « quelle est la valeur de cette variable avant la ligne ? »
- « que produit exactement `range` ? »
- « s'agit-il d'un compteur ou d'un accumulateur ? »
- « que renvoie la fonction ? »
- « quel test manque ? »
- « que se passe-t-il pour une liste vide ? »
- « le code modifie-t-il la liste d'origine ? »
- « quel est le coût quand la liste double ? »

7. Protocole de correction d'un bug

1. reproduire le bug ;
2. réduire l'exemple ;
3. identifier le premier état incorrect ;
4. formuler une hypothèse ;
5. ajouter un affichage temporaire ou une assertion ;
6. corriger une seule chose ;
7. relancer tout le jeu de tests ;
8. retirer les affichages de débogage inutiles.

8. Critères de fin de stage

- aucun mélange entre affectation et comparaison ;
- conditions et négations exactes ;
- boucles sans erreur de borne ;
- fonctions avec `return` adapté ;
- listes indexées à partir de zéro ;
- tests normaux et limites ;
- code organisé en fonctions ;
- import CSV et filtrage opérationnels ;
- présentation orale structurée.